Definition 3.6. *Let θ be the bit-string encoding the weights of a pre-trained model and $\mathtt{T}$ be a task. We say model θ is* ***(b, δ)-adaptable*** *to task $\mathtt{T}$ if:*

$$\mathrm{C}(\mathtt{T}_\delta \mid \theta) \leq b \tag{5}$$

Given this definition, we can now define the SAH as follows:

Definition 3.7. *(Informal) Let θ be a pre-trained model, b a relatively small program size, and δ a relatively high performance. The* ***superficial alignment hypothesis*** *argues that there exist many complex tasks $\mathtt{T}$ (with high $\mathrm{C}(\mathtt{T}_\delta)$) of practical interest, to which model θ is (b, δ)-adaptable.*

This grounds our intuitions that we can easily adapt a pre-trained language model for many tasks, leveraging the information already present in θ. However, our definition has one informal point: what are *relatively low* b and *relatively high* δ? Is a program of $1k$ bits and 90% accuracy superficial? Since there does not seem to be a principled way of defining a single budget–performance value (b, δ) that we would call 'superficial', we study the tradeoff between these parameters and abstain from choosing a single target value to analyse.

3.3. Connection to previous work

Our notion of Task Complexity is inspired by classical ideas from algorithmic information theory. In particular, Kolmogorov complexity (Kolmogorov, 1965) measures the information content of a finite string y as the length of the shortest program[4] that outputs y:

$$\mathrm{C}(y) \stackrel{\text{def}}{=} \min_{\mathsf{P} \in \mathcal{P}} \{\mathrm{len}(\mathsf{P}) : \mathsf{P}() = y\} \tag{6}$$

Kolmogorov complexity, however, is generally not well suited to study the complexity of machine learning (ML) tasks. First, a task $\mathtt{T}$ includes a distribution p over a potentially infinite set of inputs $\mathcal{X}$, rather than being defined as a single finite string. Second, ML is inherently concerned with approximate solutions: achieving 90% accuracy (or even 99%) in a task $\mathtt{T}$ may require far less information than achieving 100%, yet Kolmogorov complexity does not naturally express this performance-complexity tradeoff.

Algorithmic rate-distortion theory (Vereshchagin & Vitányi, 2010) addresses the second limitation above by allowing approximate outputs, defining complexity as the length of the shortest program which produces an output within a given distortion level of a target value. Interpreting this target to be an input x, and the distortion to be measured via a scoring function $\mathcal{S}$, we get:

$$\mathrm{C}(x_\delta) \stackrel{\text{def}}{=} \min_{\mathsf{P} \in \mathcal{P}} \{\mathrm{len}(\mathsf{P}) : \mathcal{S}(x, \mathsf{P}()) \geq \delta\} \tag{7}$$

[4]Note that, in this definition, the program takes no input value.

As can be seen, this equation measures the complexity of representing a fixed input x with a specific accuracy, and it thus does not address the first limitation above. Task complexity adapts Equation (7) by replacing this single-target distortion with a 'distortion' metric defined over the entire input space $\mathcal{X}$. We thus say that task complexity generalises these two notions of complexity, being more suited to the study of machine learning tasks (see Remark A.4).

Task complexity is also related to the minimum description length (MDL) principle (Grünwald, 2007) and, more specifically, to prior probing methods inspired by it (Voita & Titov, 2020; Pimentel et al., 2020a; Pimentel & Cotterell, 2021). These methods measure how compactly a dataset can be compressed given a pretrained model, and are typically used to analyse how easy-to-access is some information encoded in the model's representation.[5] In contrast, task complexity measures the length of the entire adaptation program required to achieve a target performance on the task. Also, by measuring program complexity rather than data description length alone, our framework lets us compare fundamentally different views of the SAH (e.g., inference-time control vs. data driven fine-tuning) within a single unified metric.

4. Estimating Task Complexity

We wish to compute the (b, δ)-adaptability of pre-trained models θ to common tasks $\mathtt{T}$. However, the conditional task complexity $\mathrm{C}(\mathtt{T}_\delta \mid \theta)$, on which adaptability depends, is uncomputable.[6] Hence, we upper bound this conditional complexity by finding programs P which adapt model θ to perform a task; after measuring this program's performance δ, we know that its length will naturally upper bound the task's complexity $\mathrm{C}(\mathtt{T}_\delta \mid \theta) \leq \mathrm{len}(\mathsf{P})$.

To achieve a tight upper bound, we should look for short programs which we expect to achieve high performance. How do we select such programs? We do that by leveraging the connection between our definition of task complexity and the three views of superficiality reviewed in Section 2. Each of these views represents different strategies for how to construct short, high-performance programs to adapt pre-trained models. These strategies typically leverage training data $\mathcal{D}$ drawn from task $\mathtt{T}$, and generate programs P that adapt the model to perform well on $\mathtt{T}$. Importantly, we only measure the length of the final program P output by these strategies, and not the size of the procedure generating it.

[5]Note that, due to the injectivity of transformers with real-valued hidden representations outputs (Sutter et al., 2025; Nikolaou et al., 2026), an LLM's representation always preserve all the information encoded in its inputs (Pimentel et al., 2020b). For this reason, prior work has focused on "easy-to-access information", as opposed to the total information amount.

[6]Our definition extends Kolmogorov complexity which is uncomputable (see Corollary A.5).

```
from transformers import ...

compressed_data = [0.21, 1.52, ...]

for batch in compressed_data:
  batch = decompress(batch, model)
  model(batch).loss.backward()
  optimizer.step()

output = model(eval_input)
```

(a) Data methods

```
from transformers import ...

adapter_weights = [
    0.3425,
    2.5145,
    ...
]
model.add_adapter(adapter_weights)

output = model(eval_input)
```

(b) Parametric methods

```
from transformers import ...

compressed_prompt = 0.124512213415
prompt = decompress(
    compressed_prompt,
    model
)

full_input = prompt + eval_input
output = model(full_input)
```

(c) Inference-control methods

Figure 2. Pseudo-code of the programs P constructed by strategies in Section 4. Each program includes its compressed data or parameters, and its size is usually dominated by such terms. We explain the details of how we measure program size in our experiments in Appendix B.

Together, the length (b) and performance (δ) of these programs P allow us to estimate the points on a *Pareto-optimal* length–performance (b, δ) curve, which will constitute our adaptability estimates. We now describe three procedures for constructing programs, one for each view from Section 2.

① **Data methods.** This procedure assumes access to a dataset $\mathcal{D}$ available for this task, and the pre-trained model θ. It then draws a subset $\mathcal{D}' \subseteq \mathcal{D}$ of the data and trains the model on this data. Before performing a gradient step on a target batch, however, it uses the model θ to compress this batch via arithmetic coding (Deletang et al., 2024) and writes this compressed data $\mathcal{D}'_c$ into program P. Program P then iteratively uses the model θ to decompress the data, and to update the model, being able to reconstruct such a fine-tuning procedure. Finally, the program then uses this fine-tuned model to perform inference on evaluation examples. The pseudo-code for this resulting program P is shown in Figure 2a. The length of program P then mostly depends on the size of the compressed data $\text{len}(\mathcal{D}'_c)$, with a constant overhead for, e.g., the size of the code used to decompress and train on this data. We term this procedure as **Subset Training**. The special case where the full dataset is used is labeled here as **Full Dataset**, which we use as a baseline for this approach.

② **Parametric methods.** This procedure again assumes access to a dataset $\mathcal{D}$ and the pre-trained model θ. It then first randomly initialises the weights of an adapter θ_{ADAPT} for model θ. These adapter weights θ_{ADAPT} are then trained using the data in $\mathcal{D}$, and written into a program P. The pseudo-code for these programs P is shown in Figure 2b. Notably, the program P directly encodes the adapter weights, which it uses to perform inference over the evaluation data. The program P's length thus mostly depends on the size of these adapter parameters θ_{ADAPT}, with a constant overhead for the size of the code to apply the adapter and run the model. We use two variations of this procedure, which differ in how to design and train the adapter θ_{ADAPT}. The first, **LoRA** (Hu et al., 2022), uses a number of low-rank adapter layers when fine-tuning a model. The second, **Bayesian-LoRA** (Meo et al., 2024), is similar, but directly optimises the ranks (number of parameters) and quantisation levels (precision of each parameter) in each LoRA layer, allowing a model to be adapted with fewer bits. We also consider adapting the full model as a baseline, which we label as **Full Model**.

③ **Inference-control methods.** As before, this procedure assumes access to dataset $\mathcal{D}$ and model θ. The procedure then draws a very small set of samples $\mathcal{D}' \subseteq \mathcal{D}$ and combines them into a prompt κ, which it compresses into κ_c using arithmetic coding (using model θ) and writes into a program P. Program P then decodes κ and adds it into the context of model θ before running inference on the evaluation examples. The pseudo-code for program P is shown in Figure 2c. For this strategy, the length of program P mostly depends on the size of compressed prompt $\text{len}(\kappa_c)$, with a constant overhead for the the size of the code to decompress the prompt and run the model. We experiment with two variations of this procedure. First, **In-Context Learning** (ICL; Brown et al., 2020), where the prompt κ contains examples from $\mathcal{D}'$. Second, **URIAL** (Lin et al., 2024), where an explanation of the task T is added to prompt κ in addition to the examples in $\mathcal{D}'$. Finally, we also use an empty prompt κ as a baseline, which we label as **Base Model**.

5. Experiments

Our experiments estimate the length–performance tradeoff of the (b, δ)-adaptability of real models θ on real tasks T.

5.1. Experimental setup

Universal Turing machine. Modern general-purpose programming languages are turing complete, and therefore we will define our machine M to be a python interpreter. We define all python libraries to be part of the machinery of the system, and therefore the code of these libraries themselves are not considered in the length of a program P. This means that to measure the length of a program $\text{len}(\text{P})$, we need to measure the length in bits of the python script, and any of the model weights and data used in the script.

Models. As our models θ, we use SmolLM3 3B (Bakouch et al., 2025), Olmo3 7B, and Olmo3 32B (Olmo et al., 2025).

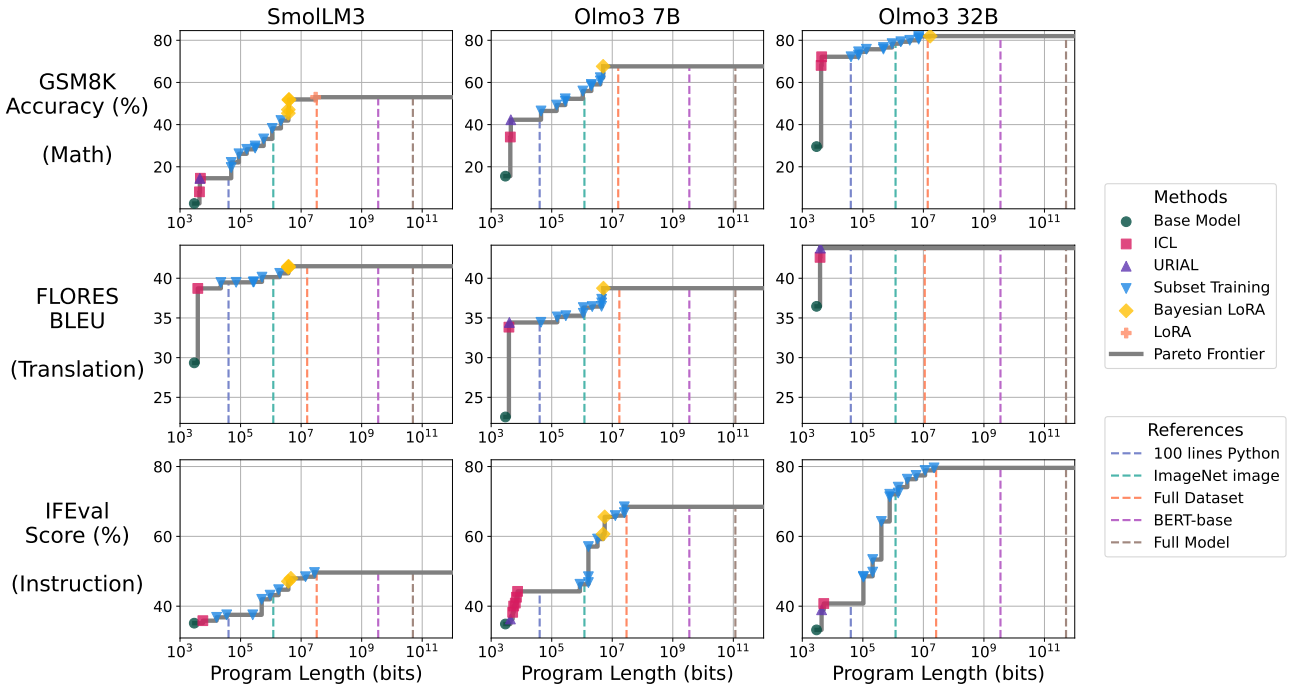

Figure 3. The program length vs. performance Pareto curves for SmolLM3 and Olmo3 in our three analysed tasks: mathematical reasoning, machine translation, and instruction following. We test each method described in Section 4, but show only the optimal lengths and performances. To provide some intuition about these program lengths, we also mark the size in bits of more commonly known references.

These models provide a wide coverage of model size and are open-source, providing not only weights but also their training data and code. For most of our experiments, we will use as our pre-trained models θ, more specifically, the last checkpoint of these models' first training stage, i.e., the last checkpoint at `stage-1` of the SmolLM3 and the Olmo3 training pipelines, respectively.

Tasks. We select three diverse NLP tasks for our experiments, which have been extensively studied in prior work: Mathematical reasoning (GSM8K; Cobbe et al., 2021),[7] English to French machine translation (FLORES-200; Costa-Jussà et al., 2024), and instruction following (IFEval; Zhou et al., 2023b). We evaluate performance using accuracy, BLEU, and verified success rate, respectively. Additional details about these tasks are provided in Appendix C.

Hyperparameters. For every method described in Section 4, we perform a hyperparameter sweep to obtain a set of length–performance points (b, δ). Based on these points, we construct a Pareto optimal curve by selecting all the (b, δ) pairs which are not dominated by any other, i.e., pairs for which no other pair performs at least as well in terms of both length and accuracy. We provide a more detailed description of these hyperparameter sweeps in Appendix D.

[7]We draw training examples from MetaMath (Yu et al., 2024), which extends the training set of GSM8K.

6. Results

Figure 3 shows our estimates of the Pareto curve between program length and task performance (b, δ) for the three analysed tasks and models. In the following, we discuss our main findings.

6.1. Evidence for superficial alignment to complex tasks

It is possible to adapt pre-trained LLMs to achieve high performances using surprisingly short programs. On GSM8K (first row of Figure 3), all pre-trained models achieve low accuracy between 2.6% and 29.6%. However, a program of only 4358 bits is sufficient to increase OLMo3-32B's accuracy to 72.2%. Similar trends hold for machine translation (second row) and instruction following (third row). The pre-trained OLMo3-7B achieves 22.63 BLEU score on English to French translation, but can be adapted to achieve 34.43 BLEU with a program of only 3992 bits. On IFEval, maximum performance is achieved with all models with programs only slightly longer than 10^7 bits (1.25MB). While these pre-trained models used terabytes of data during pre-training (see Appendix E), adapting them to achieve high performance on these tasks can be done using only megabytes of information. We interpret these results as evidence in favour of the superficial alignment hypothesis.